

Advanced SQL & Production Transformation Patterns

Target Audience: Data Engineers moving beyond standard queries to architect robust, idempotent, and high-performance ELT/ETL transformation pipelines in modern relational and analytical engines (PostgreSQL, DuckDB, Snowflake, BigQuery).

Table of Contents

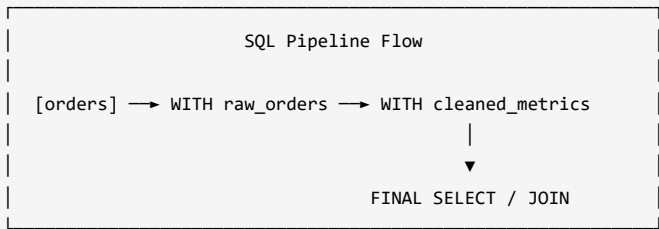
- 1. Common Table Expressions (CTEs) & The Materialization Fence
- 2. Recursive CTEs: Hierarchies, Graphs & Pathing
- 3. Correlated Subqueries & Query Transformation Mechanics
- 4. Semi-Joins & Anti-Joins: EXISTS vs NOT EXISTS vs IN
- 5. Set Operators & Automated Data Reconciliation
- 6. Conditional Aggregation & Matrix Pivoting
- 7. The SQL Standard MERGE Statement
- 8. Idempotent Upsert Patterns (ON CONFLICT)
- 9. Slowly Changing Dimensions (SCD Types 0, 1, 2, 3)
- 10. Capstone Project: Hierarchical Rollup & Incremental Upsert
- 11. Master Cheat Sheet & Optimization Rules

1. Common Table Expressions (CTEs)

A **Common Table Expression (CTE)** defines a temporary, named result set that exists only within the execution scope of a single SQL statement (SELECT , INSERT , UPDATE , DELETE).

Introduced in SQL:1999 as the WITH clause, CTEs serve two primary engineering functions:

- 1. **Pipeline Modularity:** Converting nested, unreadable "onion" subqueries into top-to-bottom procedural pipelines.
- 2. **Reusability:** Referencing the same intermediate dataset multiple times in downstream joins without re-writing logic.



Basic Syntax & Anatomy

```
WITH regional_sales AS (  
    SELECT  
        w.country,  
        COUNT(o.order_id) AS total_orders,  
        SUM(o.total_amount) AS gross_revenue  
    FROM orders.orders o  
    JOIN customer.customers c ON o.customer_id = c.customer_id  
    JOIN customer.addresses w ON c.customer_id = w.customer_id AND w.is_default = TRUE  
    WHERE o.order_status = 'delivered'  
    GROUP BY w.country  
)  
,  
top_regions AS (  
    SELECT  
        country,  
        gross_revenue,  
        RANK() OVER (ORDER BY gross_revenue DESC) AS revenue_rank  
    FROM regional_sales  
)  
SELECT  
    country,  
    gross_revenue,  
    revenue_rank  
FROM top_regions  
WHERE revenue_rank <= 5;
```

The PostgreSQL 12+ Materialization Fence: MATERIALIZED vs NOT MATERIALIZED

A critical data engineering nuance: **How does the query optimizer treat a CTE?**

Historically in PostgreSQL (v11 and below):

- CTEs were strict **optimization fences**.
- The planner evaluated the CTE completely in memory or temp disk (spooling), ignoring predicates pushed down in the outer query.

Since **PostgreSQL 12**, CTEs are inlined by default if:

1. The CTE is non-recursive and has no side-effects (`INSERT` / `UPDATE` / `DELETE`).
2. The CTE is referenced only **once** in the outer query.

You can explicitly control this behavior using `MATERIALIZED` or `NOT MATERIALIZED` :

```
-- 1. NOT MATERIALIZED (Default if single reference):
-- Planner inlines the CTE into the main query; pushdown predicates apply!
WITH filtered_orders AS NOT MATERIALIZED (
    SELECT order_id, customer_id, order_timestamp, total_amount
    FROM orders.orders
)
SELECT *
FROM filtered_orders
WHERE order_timestamp >= '2026-01-01'; -- Predicate pushes directly into table scan!

-- 2. MATERIALIZED:
-- Forces the engine to materialize the CTE as a temporary worktable once.
-- Prevents re-computing an expensive calculation when referenced multiple times!
WITH heavy_aggregation AS MATERIALIZED (
    SELECT
        product_id,
        SUM(line_total) AS total_revenue,
        AVG(quantity) AS avg_qty_per_order
    FROM orders.order_items
    GROUP BY product_id
)
SELECT
    p.product_name,
    h.total_revenue,
    h.avg_qty_per_order
FROM heavy_aggregation h
JOIN catalog.products p ON h.product_id = p.product_id
WHERE h.total_revenue > 10000;
```

Engineering Trade-Off Matrix

Feature	NOT MATERIALIZED (Inlined)	MATERIALIZED (Cached Worktable)
Predicate Pushdown	✔ Full pushdown across boundaries	✘ Blocked (filter evaluated after materialization)
Multiple References	✘ Re-executes the subquery per call	✔ Evaluates once, reads from memory cache
Index Usage	✔ Can use parent/underlying indexes	✘ No indexes on temporary CTE result set
Best Used For	Code organization, modular transformations	Expensive aggregations used in multiple joins

2. Recursive CTEs

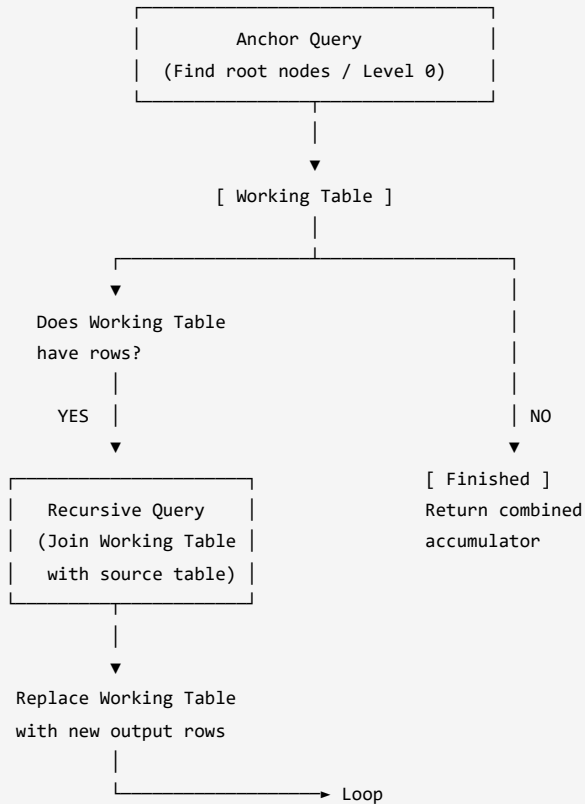
A **Recursive CTE** references itself in its own definition. It allows SQL to solve problems that traditionally required procedural looping:

- Tree traversals (Organization charts, product category taxonomies)
- Graph pathfinding (Flight routes, social connections, supply chain dependency trees)
- Sequential series generation (Date dimensions, gap-filling missing periods)

The Execution Engine Mental Model

A recursive CTE consists of three mandatory parts:

1. **Anchor Member**: The non-recursive base query that produces the initial result set (Level 0).
2. **Recursive Member**: The query that joins the recursive CTE name to another table, producing Level $N + 1$ from Level N .
3. **UNION ALL** or **UNION**: The combiner that chains each iteration until an iteration yields zero rows.



Case Study: Category Hierarchy Traversal

In our schema (`catalog.categories`), categories can have a `parent_category_id`.

Let's build a hierarchical breadcrumb path, tree depth counter, and find all children under 'Electronics':

```
WITH RECURSIVE category_tree AS (
  -- 1. Anchor Member: Top-level root categories (no parent)
  SELECT
    category_id,
    category_name,
    parent_category_id,
    1 AS depth_level,
    category_name::TEXT AS path_breadcrumbs
  FROM catalog.categories
  WHERE parent_category_id IS NULL

  UNION ALL

  -- 2. Recursive Member: Join children to their parents in the CTE
  SELECT
    c.category_id,
    c.category_name,
    c.parent_category_id,
    ct.depth_level + 1 AS depth_level,
    ct.path_breadcrumbs || ' > ' || c.category_name AS path_breadcrumbs
  FROM catalog.categories c
  INNER JOIN category_tree ct ON c.parent_category_id = ct.category_id
)
SELECT
  category_id,
  category_name,
  depth_level,
  path_breadcrumbs
FROM category_tree
ORDER BY path_breadcrumbs;
```

Preventing Infinite Loops & Cycle Detection

In real-world data, bad operational data can introduce cycles (e.g., Category A → Category B → Category A). Without protection, a recursive CTE runs until memory or disk fills up.

Technique A: Cycle Detection with Arrays (Universal SQL)

```
WITH RECURSIVE category_cycle_safe AS (  
    SELECT  
        category_id,  
        category_name,  
        parent_category_id,  
        1 AS depth,  
        ARRAY[category_id] AS visited_path,  
        FALSE AS is_cycle  
    FROM catalog.categories  
    WHERE parent_category_id IS NULL  
  
    UNION ALL  
  
    SELECT  
        c.category_id,  
        c.category_name,  
        c.parent_category_id,  
        cs.depth + 1,  
        cs.visited_path || c.category_id,  
        c.category_id = ANY(cs.visited_path) AS is_cycle  
    FROM catalog.categories c  
    JOIN category_cycle_safe cs ON c.parent_category_id = cs.category_id  
    WHERE NOT c.category_id = ANY(cs.visited_path) -- Stop recursion if cycle hit!  
        AND cs.depth < 20 -- Hard safety guardrail  
)  
SELECT * FROM category_cycle_safe;
```

Technique B: Native `CYCLE` Clause (PostgreSQL 14+)

```
WITH RECURSIVE category_tree AS (  
    SELECT category_id, category_name, parent_category_id  
    FROM catalog.categories  
    WHERE parent_category_id IS NULL  
    UNION ALL  
    SELECT c.category_id, c.category_name, c.parent_category_id  
    FROM catalog.categories c  
    JOIN category_tree ct ON c.parent_category_id = ct.category_id  
)  
CYCLE category_id SET is_cycle USING path_trace  
SELECT * FROM category_tree;
```

3. Correlated Subqueries

An **uncorrelated subquery** is independent and runs once for the entire query.

A **correlated subquery** references columns from the outer query table, requiring the conceptual evaluation of the subquery **for every single candidate row** emitted by the outer query.

```
-- Uncorrelated: Inner query runs ONCE
SELECT product_name, cost_price
FROM catalog.products
WHERE cost_price > (SELECT AVG(cost_price) FROM catalog.products);

-- Correlated: Inner query evaluates for each outer row 'p'
SELECT
    p.product_id,
    p.product_name,
    p.category_id,
    p.cost_price
FROM catalog.products p
WHERE p.cost_price > (
    SELECT AVG(sub.cost_price)
    FROM catalog.products sub
    WHERE sub.category_id = p.category_id -- Correlation link!
);
```

The Performance Hazard & Rewriting to Window Functions

Correlated subqueries in `WHERE` or `SELECT` clauses frequently manifest as **Nested Loop Scans** with $O(M \times N)$ execution complexity.

```
-- SLOW: Correlated Subquery in SELECT ( $O(M \times N)$  scans on orders)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    (
        SELECT MAX(o.order_timestamp)
        FROM orders.orders o
        WHERE o.customer_id = c.customer_id
    ) AS last_order_date
FROM customer.customers c;

-- FAST: Left Join with Pre-Aggregated CTE or Window Function ( $O(M + N)$  with Hash Aggregate)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    lo.last_order_date
FROM customer.customers c
LEFT JOIN (
    SELECT customer_id, MAX(order_timestamp) AS last_order_date
    FROM orders.orders
    GROUP BY customer_id
) lo ON c.customer_id = lo.customer_id;
```

4. EXISTS and NOT EXISTS

In relational algebra:

- `EXISTS` evaluates a **Semi-Join**: Returns rows from table A if at least one matching row exists in table B .
- `NOT EXISTS` evaluates an **Anti-Join**: Returns rows from table A if zero matching rows exist in table B .

The Short-Circuit Advantage

`EXISTS` stops scanning table B as soon as it finds the **first** qualifying record. It never evaluates the entire candidate set, nor does it generate duplicate rows like a poorly formulated `INNER JOIN` might.

```
-- Pattern: Customers who placed at least one delivered order over $500
SELECT c.customer_id, c.first_name, c.email
FROM customer.customers c
WHERE EXISTS (
  SELECT 1
  FROM orders.orders o
  WHERE o.customer_id = c.customer_id
        AND o.order_status = 'delivered'
        AND o.total_amount > 500
);
```

 Note

Writing `SELECT 1`, `SELECT *`, or `SELECT 'X'` inside `EXISTS(...)` produces identical execution plans in modern SQL optimizers. The engine ignores the projection list and solely inspects the boolean existence condition.

The Deadly `NOT IN` vs `NOT EXISTS` Trap (Three-Valued Logic)

One of the most dangerous production bugs in SQL occurs when using `NOT IN` against a subquery that contains a `NULL`.

Suppose we want customers who have never placed an order:

```
-- ⚠️ DANGEROUS: If orders.customer_id contains even ONE NULL, this returns 0 ROWS!
SELECT customer_id, email
FROM customer.customers
WHERE customer_id NOT IN (
  SELECT customer_id FROM orders.orders
);
```

Why does this happen?

In SQL three-valued logic:

$$val \notin \{1, 2, NULL\} \iff (val \neq 1) \text{ AND } (val \neq 2) \text{ AND } (val \neq NULL)$$

Since $(val \neq NULL)$ evaluates to `UNKNOWN`, the entire boolean expression resolves to `UNKNOWN` or `FALSE`. The database drops all rows!

The Bulletproof Solution: `NOT EXISTS`

`NOT EXISTS` checks whether the subquery returns **zero rows**. It evaluates to `TRUE` even if the underlying table contains nulls!

```
-- ✅ PRODUCTION-SAFE: Always returns correct anti-join results
SELECT c.customer_id, c.email
FROM customer.customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders.orders o
  WHERE o.customer_id = c.customer_id
);
```

Comparison Matrix

Approach	Handles <code>NULL</code> s Safely?	Plan Type	Index Friendly?	Recommendation
<code>NOT IN</code>	❌ Fails silently if <code>NULL</code> present	Hash / Nested Loop	Sometimes	Avoid in production
<code>NOT EXISTS</code>	✅ 100% Safe	Hash Anti-Join	✅ Excellent	Standard Best Practice
<code>LEFT JOIN ... IS NULL</code>	✅ Safe	Hash Anti-Join	✅ Good	Good, but more verbose

5. Set Operators

Set operators combine the rows of two or more queries into a single unified result.

```
Query A: [1, 2, 3, 3]
Query B: [3, 4, 5]

UNION:      [1, 2, 3, 4, 5]      (Distinct union, expensive sort)
UNION ALL:  [1, 2, 3, 3, 3, 4, 5] (Concatenation, instant)
INTERSECT:  [3]                  (Common elements only)
EXCEPT:   [1, 2]                (Elements in A but not in B)
```

Syntax and Strict Rules

- Every query in the set operation must have the **exact same number of columns**.
- Corresponding column data types must be **compatible** (e.g., `INT` with `BIGINT`, or explicit cast).
- The final column names are determined strictly by the **first** query.
- An `ORDER BY` clause can only appear once at the very end of the entire statement.

UNION ALL vs UNION (Performance Law)

- `UNION ALL` merely appends streams together. It performs no deduplication, requires no sorting, and runs in $O(N)$ time.
- `UNION` appends streams and then executes a deduplication pass (Sort Unique or Hash Aggregate), which is an $O(N \log N)$ operation that can easily spill to disk on large datasets.

 **Tip**

Data Engineer Rule: Always use `UNION ALL` unless business logic strictly mandates deduplication. If deduplication is needed across specific keys, preferred practice is explicit `ROW_NUMBER()` or `GROUP BY` to maintain deterministic control.

Automated Data Reconciliation Pattern (EXCEPT / UNION ALL)

A daily task in data engineering is **reconciling staging vs production tables** during migration or CDC verification:

```
-- Find rows that differ between Staging and Production tables:
(
  SELECT sku, product_name, cost_price, status FROM catalog.products_staging
  EXCEPT
  SELECT sku, product_name, cost_price, status FROM catalog.products
)
UNION ALL
(
  SELECT sku, product_name, cost_price, status FROM catalog.products
  EXCEPT
  SELECT sku, product_name, cost_price, status FROM catalog.products_staging
);
-- If this query returns 0 rows, the datasets are 100% synchronized!
```

6. Conditional Aggregation

Conditional aggregation evaluates expressions conditionally inside aggregate functions. It is the cornerstone of:

- Pivot tables (converting row dimensions to column metrics)
- Single-pass KPI dashboards
- Cohort conversion and funnel metrics

Method A: Standard SQL `CASE WHEN`

```
SELECT
  DATE_TRUNC('month', order_timestamp) AS order_month,
  COUNT(order_id) AS total_orders,
  SUM(CASE WHEN order_status = 'delivered' THEN total_amount ELSE 0 END) AS delivered_revenue,
  SUM(CASE WHEN order_status = 'cancelled' THEN total_amount ELSE 0 END) AS cancelled_loss,
  ROUND(
    100.0 * COUNT(CASE WHEN order_status = 'delivered' THEN 1 END) / COUNT(order_id),
    2
  ) AS delivery_success_rate_pct
FROM orders.orders
GROUP BY DATE_TRUNC('month', order_timestamp)
ORDER BY order_month;
```

Method B: Modern PostgreSQL `FILTER (WHERE ...)`

PostgreSQL supports the SQL standard `FILTER` clause on aggregates, which is cleaner, faster, and avoids `ELSE 0` or `ELSE NULL` confusion:

```
SELECT
  DATE_TRUNC('month', order_timestamp) AS order_month,
  COUNT(order_id) AS total_orders,
  SUM(total_amount) FILTER (WHERE order_status = 'delivered') AS delivered_revenue,
  SUM(total_amount) FILTER (WHERE order_status = 'cancelled') AS cancelled_loss,
  COUNT(*) FILTER (WHERE payment_method = 'upi') AS upi_orders,
  COUNT(*) FILTER (WHERE payment_method = 'credit_card') AS card_orders
FROM orders.orders o
JOIN orders.payments p ON o.order_id = p.order_id
GROUP BY DATE_TRUNC('month', order_timestamp)
ORDER BY order_month;
```

7. The `MERGE` Statement

Introduced in the SQL:2006/2016 standards and natively added to **PostgreSQL 15**, `MERGE` provides a unified syntax to perform conditional `INSERT`, `UPDATE`, and `DELETE` in a single atomic statement against a target table using a source table.

```
MERGE INTO target_table AS t
USING source_table AS s
ON t.id = s.id
WHEN MATCHED AND s.is_deleted = TRUE THEN
  DELETE
WHEN MATCHED THEN
  UPDATE SET
    t.val = s.val,
    t.updated_at = s.updated_at
WHEN NOT MATCHED THEN
  INSERT (id, val, updated_at)
VALUES (s.id, s.val, s.updated_at);
```

Real-World Example: Syncing Daily Product Price Updates

```
MERGE INTO catalog.products AS target
USING staging.product_updates AS source
ON target.sku = source.sku
WHEN MATCHED AND source.action = 'DISCONTINUE' THEN
    UPDATE SET status = 'discontinued', updated_at = NOW()
WHEN MATCHED THEN
    UPDATE SET
        cost_price = source.cost_price,
        status = source.status,
        updated_at = NOW()
WHEN NOT MATCHED THEN
    INSERT (product_id, sku, product_name, category_id, brand, cost_price, status, launch_date, created_at, updated_at)
    VALUES (gen_random_uuid(), source.sku, source.product_name, source.category_id, source.brand, source.cost_price, source.status, CUR
```

8. Upsert Patterns and Idempotency

In batch pipelines, pipelines must be **idempotent**: running the pipeline multiple times with the same input data must yield the exact same target state without creating duplicates or errors.

PostgreSQL's Native Workhorse: `INSERT ... ON CONFLICT`

While `MERGE` is versatile, PostgreSQL's `INSERT ... ON CONFLICT` (introduced in PG 9.5) remains the gold standard for high-concurrency upserts because it handles concurrency conflicts at the storage engine level without row-locking deadlocks.

```
-- Pattern 1: DO NOTHING (Idempotent Append / Deduplication)
INSERT INTO customer.customers (
    customer_id, customer_code, first_name, last_name, email,
    signup_date, country, customer_status, created_at, updated_at
)
VALUES (
    gen_random_uuid(), 'CUST-9999', 'John', 'Doe', 'john.doe@example.com',
    NOW(), 'USA', 'active', NOW(), NOW()
)
ON CONFLICT (email)
DO NOTHING;

-- Pattern 2: DO UPDATE (True Upsert with EXCLUDED virtual table)
INSERT INTO catalog.products (
    product_id, sku, product_name, category_id, brand, cost_price, status, launch_date, created_at, updated_at
)
VALUES (
    gen_random_uuid(), 'SKU-ELECT-001', 'Wireless Noise-Cancelling Headphones',
    'c0a80121-0000-0000-0000-000000000001', 'SoundMaster', 89.99, 'active', CURRENT_DATE, NOW(), NOW()
)
ON CONFLICT (sku)
DO UPDATE SET
    product_name = EXCLUDED.product_name,
    cost_price = EXCLUDED.cost_price,
    status = EXCLUDED.status,
    updated_at = NOW()
WHERE catalog.products.cost_price != EXCLUDED.cost_price
OR catalog.products.status != EXCLUDED.status; -- Skip no-op writes!
```



Tip

The `WHERE` clause at the end of `DO UPDATE` is a high-performance optimization: it prevents PostgreSQL from generating WAL logs and updating row versions (`HOT` updates) if the incoming data is identical to existing data!

MERGE VS ON CONFLICT Comparison

Feature	INSERT ... ON CONFLICT	MERGE INTO
PostgreSQL Version	9.5+	15+
Conflict Target	Requires Unique Index / Constraint	Evaluates arbitrary boolean expression
Concurrency Safety	High (handles concurrent race conditions cleanly)	Can throw serialization/concurrency errors under high load
Can Delete Rows?	❌ No (only <code>DO NOTHING</code> or <code>DO UPDATE</code>)	✅ Yes (<code>WHEN MATCHED ... THEN DELETE</code>)
Best Used For	High-throughput streaming ingest, webhooks	Complex data warehouse stage-to-target merges

9. Slowly Changing Dimensions (SCD Types 0 to 3)

In dimensional modeling (Kimball methodology), dimensions are not static; attributes change slowly over time. How your warehouse handles changes dictates historical truth and auditability.

SCD Type	Strategy	Core Storage Mechanism
Type 0	Retain Original (Immutable)	Never update (<code>ON CONFLICT DO NOTHING</code>)
Type 1	Overwrite In-Place (No History)	Overwrite existing row (<code>ON CONFLICT DO UPDATE</code>)
Type 2	Add New Row (Full History)	Expire old row (<code>valid_to</code>), insert new row (<code>is_current</code>)
Type 3	Previous & Current Attribute	Store <code>current_val</code> and <code>previous_val</code> in same row

SCD Type 0: Retain Original (Fixed / Immutable)

Used for attributes that must NEVER change once created, such as `original_signup_date` or `date_of_birth` :

```
INSERT INTO dim_customer_scd0 (customer_id, date_of_birth, acquisition_channel)
VALUES ('a0000000-0000-0000-0000-000000000001', '1995-06-15', 'organic_search')
ON CONFLICT (customer_id)
DO NOTHING; -- Incoming updates from source are rejected
```

SCD Type 1: Overwrite (Current State Only)

Used for correcting typos or attributes where historical tracking has zero analytical value (e.g. updating phone number):

```
INSERT INTO dim_customer_scd1 (customer_id, email, phone, updated_at)
VALUES ('a0000000-0000-0000-0000-000000000001', 'alice@newmail.com', '+1-555-0999', NOW())
ON CONFLICT (customer_id)
DO UPDATE SET
  email      = EXCLUDED.email,
  phone      = EXCLUDED.phone,
  updated_at = EXCLUDED.updated_at;
```

SCD Type 2: Add New Row (Full Historical Versioning)

The gold standard for data warehouses. Enables point-in-time "as-of" queries:

```

-- Step 1: Expire active record when an attribute changes
UPDATE dim_customer_scd2
SET valid_to = NOW(), is_current = FALSE
WHERE customer_id = 'a0000000-0000-0000-0000-000000000001' AND is_current = TRUE;

-- Step 2: Insert new version
INSERT INTO dim_customer_scd2 (
    customer_id, membership_tier, country, valid_from, valid_to, is_current
)
VALUES (
    'a0000000-0000-0000-0000-000000000001', 'Gold', 'USA', NOW(), '9999-12-31 23:59:59+00', TRUE
);

```

SCD Type 3: Add New Column (Previous & Current Value)

Preserves a single prior snapshot directly on the same record (e.g. tracking sales rep re-assignments):

```

UPDATE dim_customer_scd3
SET
    previous_tier      = current_tier,
    current_tier       = 'Gold',
    tier_effective_date = NOW()
WHERE customer_id = 'a0000000-0000-0000-0000-000000000001';

```

(Complete implementations and point-in-time queries in `10_scd_patterns.sql`)

10. Capstone Project

Part 1: Hierarchical Category Revenue Rollup

Calculating not just the revenue directly attached to a category, but recursively rolling up revenue from all child and grandchild subcategories to their root parents.

Part 2: Incremental Staging Upsert Pipeline

Simulating a real ELT landing table, applying data deduplication with `ROW_NUMBER()` , and executing an idempotent merge into production.

(Full implementations provided in `09_build_hierarchical_upsert.sql`)

11. Master Cheat Sheet

Pattern	Use Case	Core Syntax / Keyword
Common Table Expression	Clean pipeline staging	WITH cte_name AS (...)
Materialized CTE	Cache expensive multi-use result	WITH cte_name AS MATERIALIZED (...)
Recursive CTE	Trees, graphs, sequences	WITH RECURSIVE r AS (Anchor UNION ALL Recursive)
Correlated Subquery	Row-dependent metric	WHERE col > (SELECT AVG(c) WHERE ref = outer.ref)
Semi-Join	Check existence without duplicate	WHERE EXISTS (SELECT 1 FROM ...)
Anti-Join	Safe absence check (null-safe)	WHERE NOT EXISTS (SELECT 1 FROM ...)
UNION ALL	Fast stream concatenation	SELECT ... UNION ALL SELECT ...
Data Diff / Reconcile	Table comparison	(A EXCEPT B) UNION ALL (B EXCEPT A)
Conditional Aggregation	Single-pass pivot / KPI metrics	SUM(val) FILTER (WHERE status = 'X')
MERGE	Standard multi-action merge	MERGE INTO t USING s ON (...) WHEN MATCHED...
ON CONFLICT DO UPDATE	Concurrency-safe atomic upsert	INSERT ... ON CONFLICT (key) DO UPDATE SET col=EXCLUDED
SCD Type 0	Immutable static data	ON CONFLICT (key) DO NOTHING
SCD Type 1	In-place overwrite (no history)	ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col
SCD Type 2	True point-in-time history	valid_from, valid_to, is_current, surrogate key
SCD Type 3	Current + previous attribute	SET prev_col = curr_col, curr_col = new_val